

Laboratorio 03: Implementación de Arquitectura Lambda

Integrantes:
Felipe González,
Vianca Viveros,
Valentina Arcas,
Francisco López,
Benjamín Lagos

Asignatura: Big Data
Fecha: 04 de Octubre 2026

GitHub: https://github.com/benjitaminmon/Lab08-Big_Data

Índice

Laboratorio 03: Implementación de Arquitectura Lambda	1
Instrucciones	3
Instalación	5
Docker	6
Apache Kafka	7
Instalación en Docker	7
Test - Ingesta de Datos	8
Apache Spark y PySpark	10
1 Crear carpetas:	10
2 Configurar la descarga de Spark Downloads Apache Spark:	10
3 Entrar al link y descargar	10
4 extraer el archivo en C:\spark:	10
5 Descargar winutils de Hadoop (cdarlint/winutils: winutils.exe hadoop.dll and hdfs.dll binaries for hadoop windows):	11
6 El archivo descargado ponlo en en C:\hadoop\bin\	11
7 Configuración de variables de entorno	12
8 Instalar desde la terminal con pip install (Findspark es opcional):	14
Capa Speed	15
Magia	16
Capa Batch	18
Magia	19
CSV	20
Tabla comparativa de arquitecturas Big Data	21
Conclusión	23
Bibliografía	24

Instrucciones

Cada equipo deberá implementar un ejemplo práctico sencillo, simulando una pequeña solución Big Data en Windows, basada en una de las siguientes arquitecturas: Lambda, Kappa o Data Lakehouse (durante la clase del 21-09, el equipo debe indicar que arquitectura usará, solo se puede repetir dos arquitecturas 2 veces). Antes de decidir, los estudiantes deben leer el material complementario de la clase; este lo pueden encontrar en módulos semana 08, bajo el nombre "MaterialComplementarioArquitecturasBigData.pdf".

Este laboratorio se realizará durante las clases del 21-09 y el 28-09, quedando su entrega a través de una exposición el 05-09.

Para este taller, los equipos se generarán de forma aleatoria durante la clase del 21-09. El grupo debe generar un informe, en el que documenta el proceso agregando evidencias, se debe entregar el código (en un archivo zip). Además, el trabajo debe contemplar una tabla comparativa entre las arquitecturas. A continuación, le presentamos algunos ejemplos propuestos para el desarrollo de lo solicitado; cada equipo deberá decidir si su trabajo lo realiza en base al ejemplo o no.

Arquitectura Lambda

Pasos sugeridos:

- Instalar Apache Kafka, Apache Spark y Python (PySpark) en Windows.
- Simular la ingesta de eventos con un productor de Kafka (por ejemplo, logs cada segundo).
- Procesar los datos en dos vías:
 - Batch: Leer los datos acumulados cada X minutos con PySpark y generar un reporte CSV (por ejemplo, conteo total de logs por usuario).
 - Streaming (Speed): Procesar los datos en tiempo real con Spark Streaming sobre Kafka, mostrando alertas o resultados en consola.
- Unir resultados en una "capa de servicio" (base SQLite o carpeta CSV).
- Documentar comandos y scripts usados.

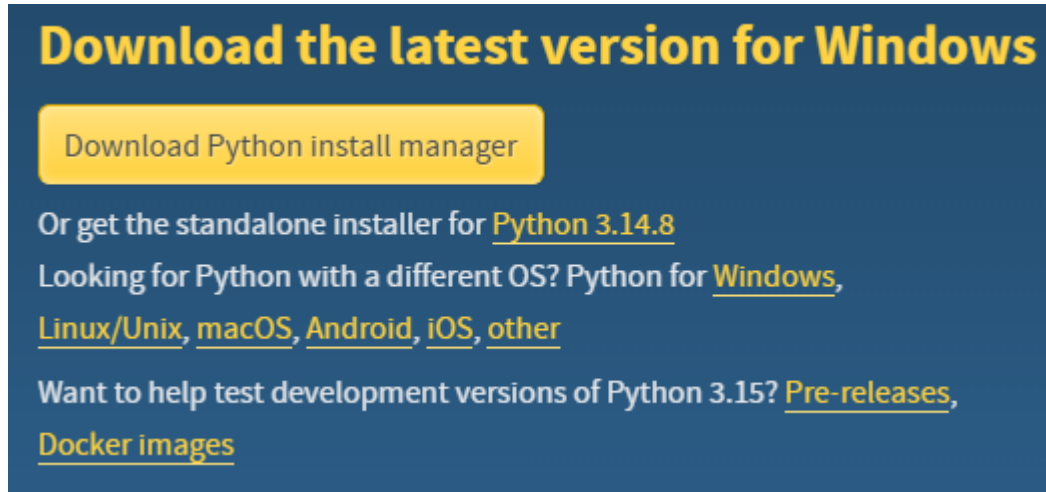
Tecnologías propuestas: Kafka, Spark (PySpark), Python, SQLite (opcional), Docker (opcional para simplificar instalación).

Criterio	Puntaje	Descripción
Funcionamiento de ejemplo	30	Pipeline simulado operativo y resultados generados
Calidad y claridad del código	15	Estructura, comentarios, modularidad
Documentación	15	Detalle de pasos, pantallazos, explicación técnica
Análisis comparativo en tabla	10	Tabla con criterios bien definidos y justificados
Presentación y exposición	10	Claridad de la defensa oral y respuestas a preguntas
Creatividad y dificultad	10	Propuesta de datos, ventanas, métricas, visualización
Resolución de problemas/tips	10	Herramientas usadas, problemas resueltos y recomendaciones
Tiempo	10	La presentación dura mínimo 12 minutos y máximo 15 puntos

Instalación

Es importante que ya hayas instalado previamente Python y Java (JDK) en la computadora.

Python: <https://www.python.org/downloads/>



JDK21: https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe

JDK 27 JDK 25 JDK 21

Java SE Development Kit 21.0.12.1 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

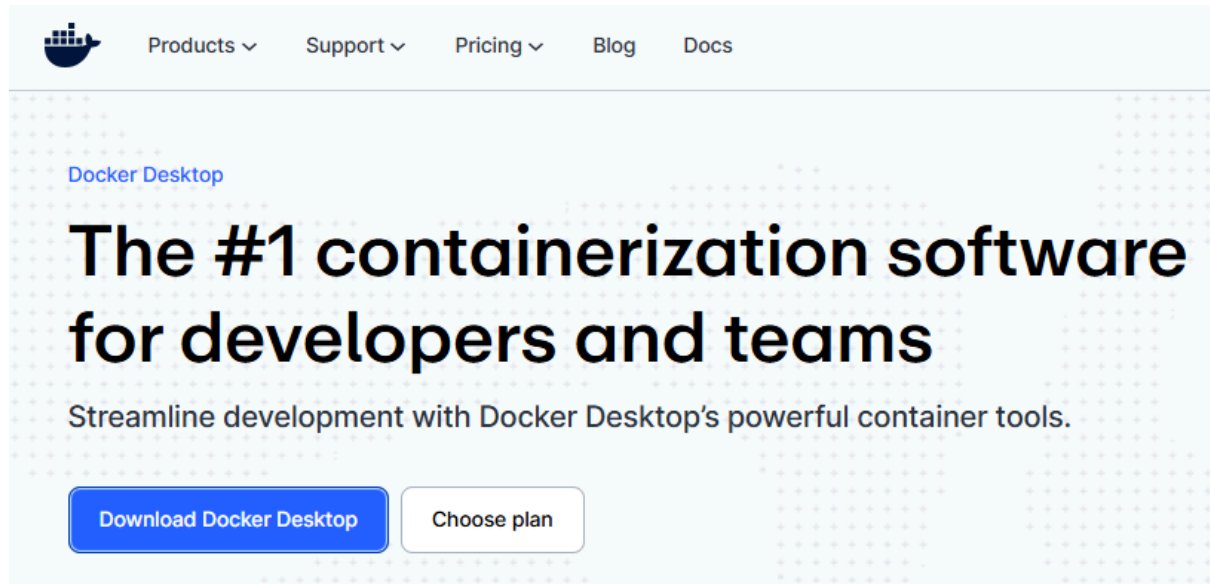
JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	187.82 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	166.94 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	165.71 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

Docker

Debido a una actualización de Apache Kafka, para el trabajo fue necesario descargar Docker Desktop para la ejecución de Kafka



Docker:

https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe?utm_source=docker&utm_medium=webreferral&utm_campaign=dd-smartbutton&utm_location=module

Apache Kafka

Instalación en Docker

Una vez creada una carpeta de proyecto, en “docker-compose.yml”, pondremos la configuración del docker, el cual creará su ambiente para usar Kafka

```

Run All Services
services:
  Run Service
  kafka:
    image: apache/kafka:latest
    container_name: servidor-kafka
    ports:
      - "9092:9092"
    environment:
      - KAFKA_NODE_ID=1
      - KAFKA_PROCESS_ROLES=broker,controller
      - KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
      - KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER
      - KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      - KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093
      - KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1
      - KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1
      - KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=1
      - KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS=0
      - KAFKA_NUM_PARTITIONS=1
      - KAFKA_AUTO_CREATE_TOPICS_ENABLE=true
    volumes:
      - ./kafka_data:/tmp/kafka-logs
  
```

Con Docker abierto y estando en la ruta del proyecto levantamos Kafka:

```

C:\Users\felip\Downloads\Laboratorio_BigData>docker compose up -d
[+] up 16/16
✓Image apache/kafka:latest          Pulled          14.3s
✓Network laboratorio_bigdata_default Created          0.1s
✓Container servidor-kafka           Started          2.3s
  
```

Comprobamos en Docker desktop que se levantó el contenedor:

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last s	Actions
☐	laboratorio_bigdata -	-	-	-	4.72%	2 min	    

Test - Ingesta de Datos

Usaremos un código en Python, “productor_kafka.py”, que se conectará a nuestro contenedor.

Nuestro sistema comienza con un script en Python que actúa como productor, simulando la actividad de distintos usuarios y empaquetando cada acción en un registro con formato JSON. Estos eventos son transmitidos continuamente hacia un canal o “topic” dentro de Apache Kafka, el cual funciona como el motor central de nuestra ingesta de datos.

A partir de este punto, la arquitectura se divide en dos vías concurrentes: Apache Spark se conecta como consumidor a este mismo topic para extraer la información, alimentando simultáneamente nuestra capa Speed para el procesamiento y monitoreo en tiempo real, y nuestra capa Batch para el almacenamiento y análisis estructurado del historial consolidado.

IMPORTANTE: Se debe instalar pip install kafka-python

```
PS C:\Users\benja\Desktop\Universidad\6to semestre-2026\Big Data\Lab08-Big_Data> pip install kafka-python
Collecting kafka-python
  Downloading kafka_python-3.0.11-py3-none-any.whl.metadata (11 kB)
  Downloading kafka_python-3.0.11-py3-none-any.whl (614 kB)
    614.2/614.2 kB 4.5 MB/s eta 0:00:00
Installing collected packages: kafka-python
  WARNING: The script kafka-python.exe is installed in 'C:\Users\benja\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed kafka-python-3.0.11

[notice] A new release of pip is available: 25.1.1 -> 26.2.1
[notice] To update, run: C:\Users\benja\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

```
productor_kafka.py > ...
1  import json
2  import time
3  import random
4  from kafka import KafkaProducer
5  from datetime import datetime
6
7  # 1. Configuración del Productor de Kafka
8  def conectar_kafka():
9      try:
10         producer = KafkaProducer(
11             bootstrap_servers=['localhost:9092'],
12             value_serializer=lambda v: json.dumps(v).encode('utf-8')
13         )
14         return producer
15     except Exception as e:
16         print(f"Error al conectar con Kafka: {e}")
17         return None
18
19 # 2. Simulación de Datos (Creatividad de dominio)
20 usuarios = ['Usuario1', 'Usuario2', 'Usuario3', 'Usuario4', 'Usuario5', 'Usuario6', 'Usuario7', 'Usuario8', 'Usuario9', 'Usuario10']
21 acciones = ['despertar', 'desayunar', 'trabajar', 'almorzar', 'dormir']
22
```

```
23 def generar_evento():
24     return {
25         'timestamp': datetime.now().isoformat(),
26         'usuario': random.choice(usuarios),
27         'accion': random.choice(acciones),
28         'tiempo_respuesta_ms': random.randint(50, 500)
29     }
30
31 # 3. Bucle de Ingesta (1 evento por segundo como pide el instructivo)
32 def iniciar_simulacion():
33     producer = conectar_kafka()
34     if not producer:
35         return
36
37     print("Iniciando productor de Kafka... Presiona Ctrl+C para detener.")
38     topic_name = 'logs-plataforma'
39
40     try:
41         while True:
42             evento = generar_evento()
43             producer.send(topic_name, value=evento)
44             print(f"Enviado a Kafka -> {evento}")
45             time.sleep(1) # Simula log cada segundo
46     except KeyboardInterrupt:
47         print("\nSimulación detenida por el usuario.")
48     finally:
49         producer.close()
50         print("Conexión con Kafka cerrada.")
51
52 if __name__ == '__main__':
53     iniciar_simulacion()
```

Prueba de ingesta de datos (Presionar Ctrl + C para terminar el proceso):

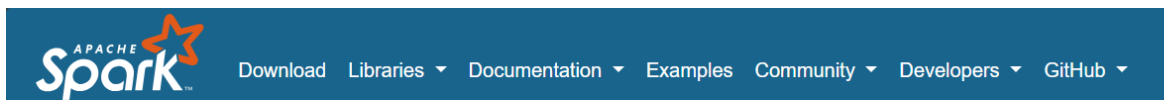
```
C:\Users\felip\Downloads\Laboratorio_BigData>python productor_kafka.py
```

Apache Spark y PySpark

1 Crear carpetas:

C:\hadoop\bin
C:\spark
C:\tmp\hive

2 Configurar la descarga de Spark [Downloads | Apache Spark:](#)



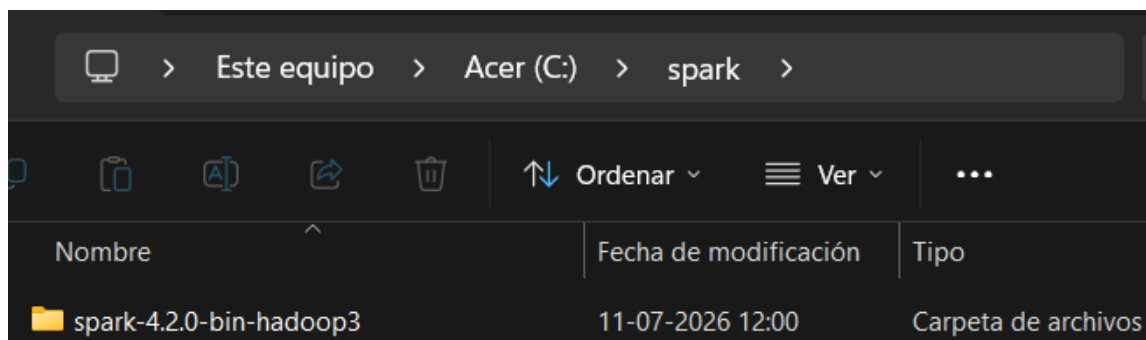
Downloads

1. Choose a Spark release:
2. Choose a package type:
3. Download Spark: [spark-4.2.0-bin-hadoop3.tgz](#)

3 Entrar al link y descargar

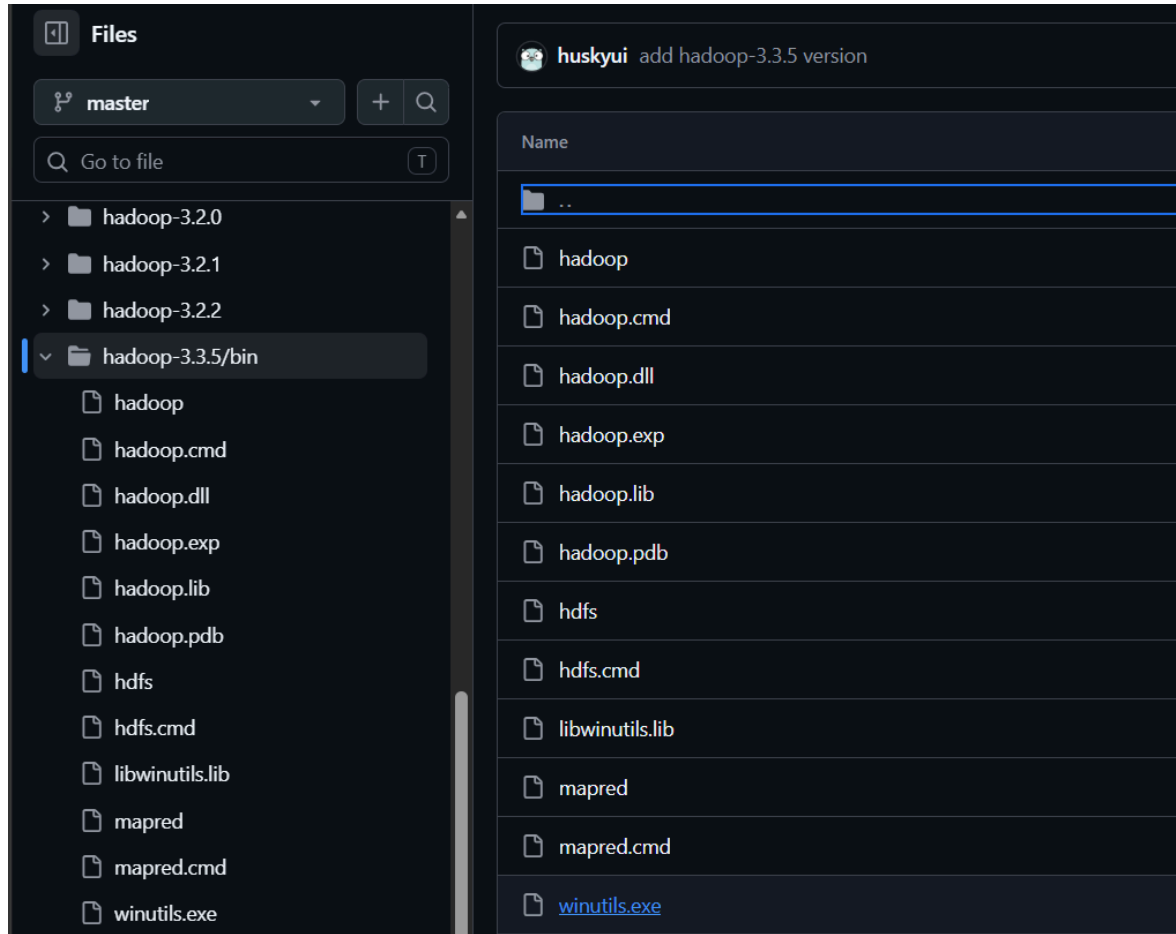


4 extraer el archivo en C:\spark:

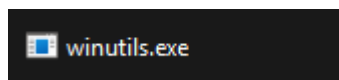


5 Descargar winutils de Hadoop

[cdarlint/winutils: winutils.exe hadoop.dll and hdfs.dll binaries for hadoop windows:](#)

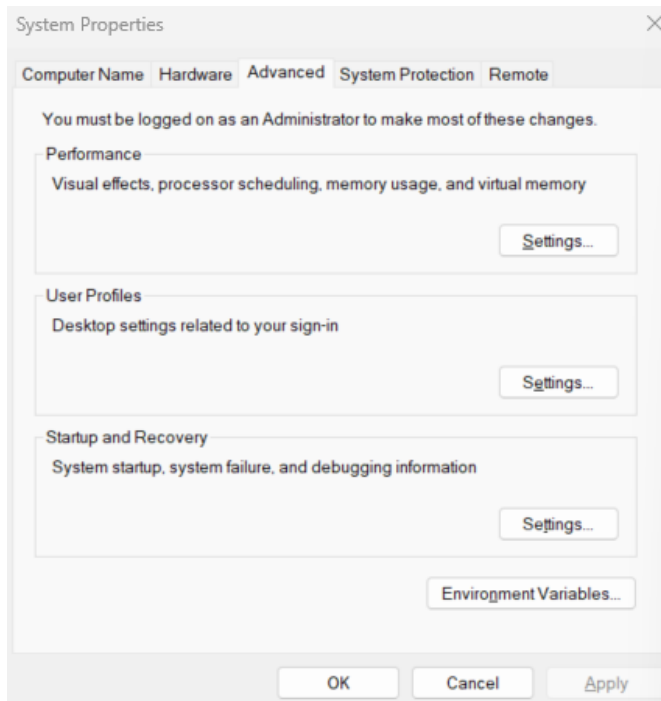


6 El archivo descargado ponlo en en C:\hadoop\bin\

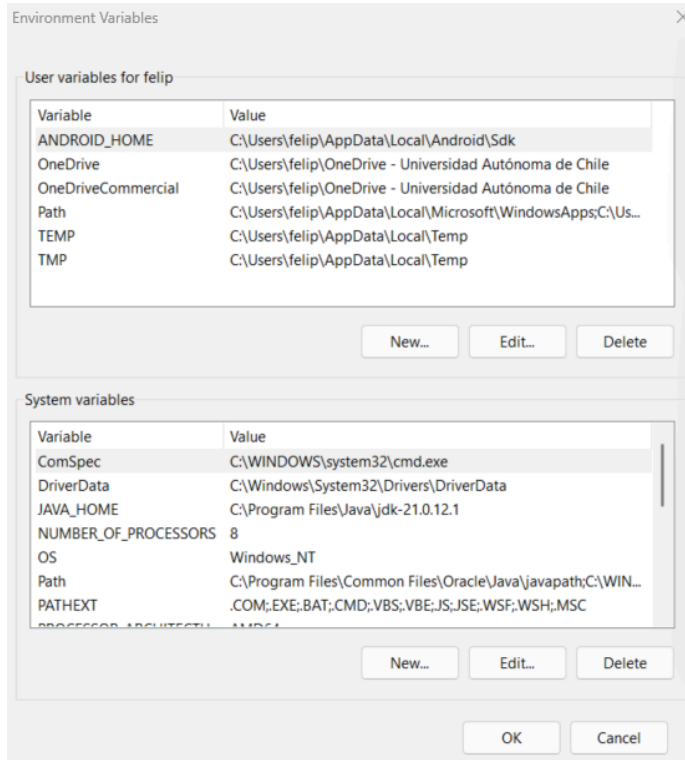


7 Configuración de variables de entorno

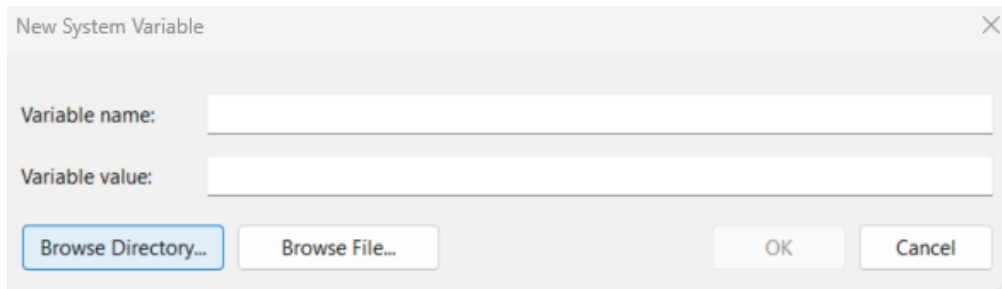
- Presiona la tecla Win y busca “Variables de entorno”:



- En “Variables de sistema” (Environment Variables...):



- Selecciona “Nuevo” (New...):

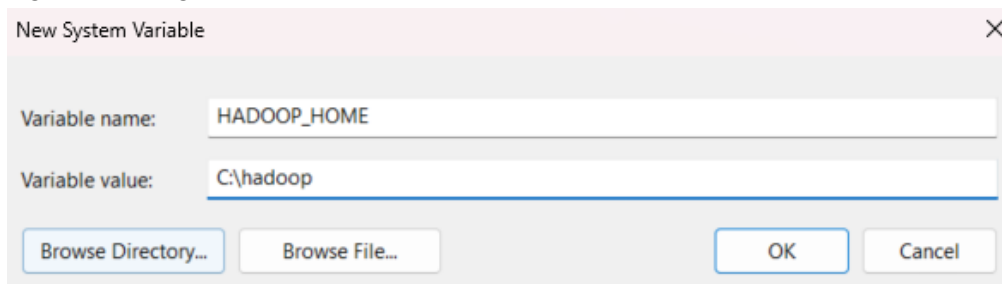


New System Variable

Variable name:

Variable value:

- Ingresa los siguientes datos:

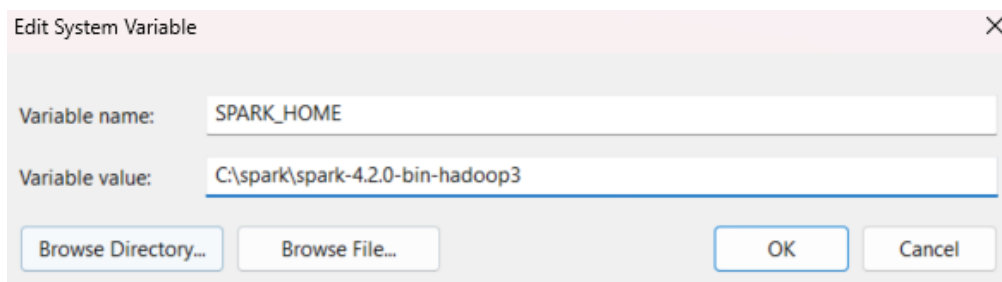


New System Variable

Variable name:

Variable value:

- Y también:

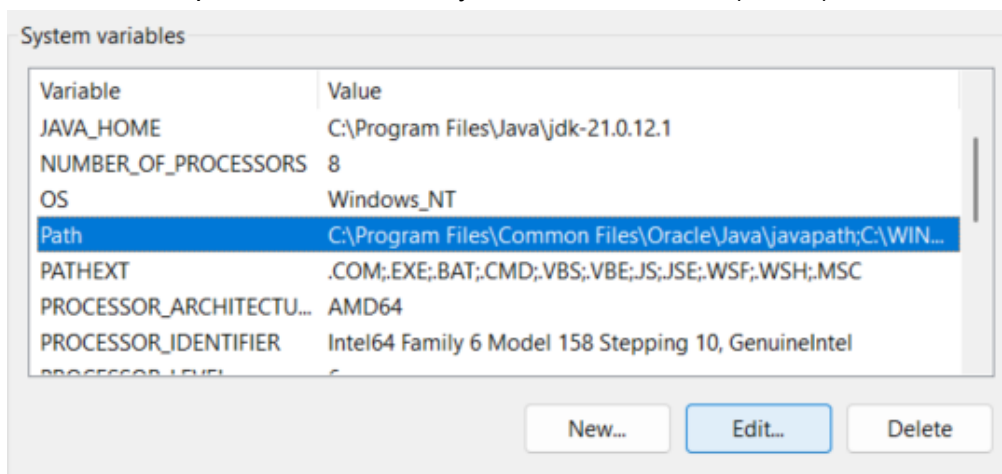


Edit System Variable

Variable name:

Variable value:

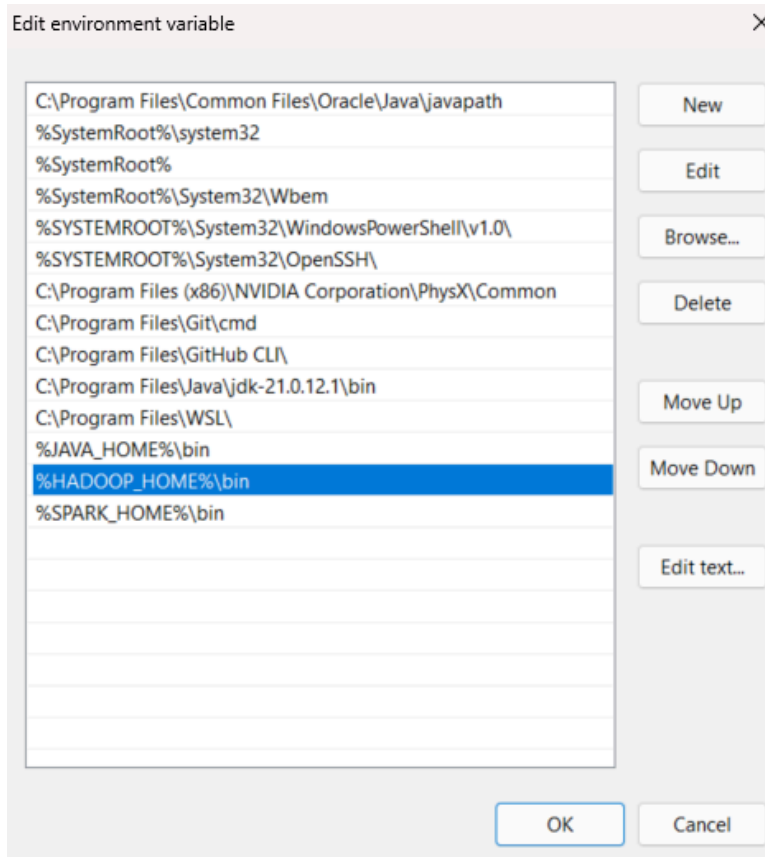
- Al salir de esa pantalla ve a “Path” y selecciona “Editar” (Edit...)



System variables

Variable	Value
JAVA_HOME	C:\Program Files\Java\jdk-21.0.12.1
NUMBER_OF_PROCESSORS	8
OS	Windows_NT
Path	C:\Program Files\Common Files\Oracle\Java\javapath;C:\WIN...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTU...	AMD64
PROCESSOR_IDENTIFIER	Intel64 Family 6 Model 158 Stepping 10, GenuineIntel
PROCESSOR_LEVEL	6

- Agrega las 2 últimas líneas cliqueando en “Nuevo” (New)



8 Instalar desde la terminal con pip install (FindSpark es opcional):

```

C:\Users\benja>pip install pyspark findspark
Collecting pyspark
  Downloading pyspark-4.2.0.tar.gz (450.1 MB)
    450.1/450.1 MB 9.8 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting findspark
  Downloading findspark-2.0.1-py2.py3-none-any.whl.metadata (352 bytes)
Collecting py4j<0.10.9.10,>=0.10.9.7 (from pyspark)
  Downloading py4j-0.10.9.9-py2.py3-none-any.whl.metadata (1.3 kB)
  Downloading py4j-0.10.9.9-py2.py3-none-any.whl (203 kB)
  Downloading findspark-2.0.1-py2.py3-none-any.whl (4.4 kB)
Building wheels for collected packages: pyspark
  Building wheel for pyspark (pyproject.toml) ... done
  Created wheel for pyspark: filename=pyspark-4.2.0-py2.py3-none-any.whl size=450798710 sha256=baef9c0e4f044b9fbb7bd687f396dc74da4d8e6a3858cb8d95805f5bba27338a
  Stored in directory: c:\users\benja\appdata\local\pip\cache\wheels\61\69\04\e7d825c33d54d9b6e7a1b94c56998f54832c38b64553603219
Successfully built pyspark
Installing collected packages: py4j, findspark, pyspark
Successfully installed findspark-2.0.1 py4j-0.10.9.9 pyspark-4.2.0

[notice] A new release of pip is available: 25.1.1 -> 26.2.1
[notice] To update, run: C:\Users\benja\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
  
```

Capa Speed

Esta capa se encargará del Streaming en tiempo real. Para esto necesitaremos un nuevo código “consumidor_speed.py”. En esta capa utilizamos la función “readStream” de PySpark para mantener una conexión abierta y continua con el topic de Kafka.

A diferencia de un análisis tradicional, el código captura los datos crudos en tiempo real, interpreta su estructura JSON y aplica una agregación en memoria. Esto permite que los conteos de acciones por usuario se actualicen instantáneamente en la consola a medida que ingresan nuevos registros.

```
1  import os
2  import findspark
3  findspark.init()
4  # Tip de resolución de problemas: Descarga automáticamente el conector de Kafka para Spark
5  os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.2.0 pyspark-shell'
6
7  from pyspark.sql import SparkSession
8  from pyspark.sql.functions import from_json, col
9  from pyspark.sql.types import StructType, StructField, StringType, IntegerType
10
11 # Iniciar sesión de Spark
12 spark = SparkSession.builder.appName("CapaSpeed_Lambda").getOrCreate()
13 spark.sparkContext.setLogLevel("ERROR") # Ocultar logs innecesarios
14
15 # Definir la estructura de nuestros datos JSON
16 esquema = StructType([
17     StructField("timestamp", StringType(), True),
18     StructField("usuario", StringType(), True),
19     StructField("accion", StringType(), True),
20     StructField("tiempo_respuesta_ms", IntegerType(), True)
21 ])
22
23 print("Conectando a Kafka en tiempo real (Capa Speed)...")
24
25 # Leer desde Kafka en modo Streaming
26 df_stream = spark.readStream \
27     .format("kafka") \
28     .option("kafka.bootstrap.servers", "localhost:9092") \
29     .option("subscribe", "logs-plataforma") \
30     .load()
31
32 # Transformar los datos binarios a formato tabular
33 df_parsed = df_stream.selectExpr("CAST(value AS STRING) as json_str") \
34     .select(from_json(col("json_str"), esquema).alias("data")).select("data.*")
35
36 # Agrupar y contar las acciones de los usuarios en tiempo real
37 df_agrupado = df_parsed.groupBy("usuario").count()
38
39 # Mostrar resultados en la consola
40 query = df_agrupado.writeStream \
41     .outputMode("complete") \
42     .format("console") \
43     .start()
44
45 query.awaitTermination()
```

Magia

- Encendemos Docker Desktop abriendo una terminal: *docker compose up -d*
- Prendemos: *python productor_kafka.py*
- En **otra terminal** arrancamos: *python consumidor_speed.py*

Con eso empezaremos a ver cómo Spark empieza a procesar los datos

Kafka:

```
C:\Users\felip\Downloads\Laboratorio_BigData>docker compose up -d
[+] up 1/1
✓Container servidor-kafka Started 0.4s

C:\Users\felip\Downloads\Laboratorio_BigData>python productor_kafka.py
C:\Users\felip\Downloads\Laboratorio_BigData\productor_kafka.py:33: DeprecationWarning: value_serializer does not implement kafka.serializer.Serializer
    producer = conectar_kafka()
Iniciando productor de Kafka... Presiona Ctrl+C para detener.
Enviado a Kafka -> {'timestamp': '2026-10-04T20:59:03.709649', 'usuario': 'Usuario3', 'accion': 'despertar', 'tiempo_respuesta_ms': 99}
```

Spark:

```
C:\Users\felip\Downloads\Laboratorio_BigData>python consumidor_speed.py
WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url = jar:file:/C:/spark/spark-4.2.0-bin-hadoop3/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: C:\Users\felip\.ivy2.5.2\cache
The jars for the packages stored in: C:\Users\felip\.ivy2.5.2\jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-d2c3aea9-4506-4903-a0f8-05358a94ead7;1.0
  confs: [default]
    found org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 in central
    found org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 in central
    found org.apache.kafka#kafka-clients;3.4.1 in central
    found org.lz4#lz4-java;1.8.0 in central
    found org.xerial.snappy#snappy-java;1.1.10.3 in central
    found org.slf4j#slf4j-api;2.0.7 in central
    found org.apache.hadoop#hadoop-client-runtime;3.3.4 in central
    found org.apache.hadoop#hadoop-client-api;3.3.4 in central
    found commons-logging#commons-logging;1.1.3 in central
    found com.google.code.findbugs#jsr305;3.0.0 in central
    found org.apache.commons#commons-pool2;2.11.1 in central
```

```
org.lz4#lz4-java;1.8.0 from central in [default]
org.slf4j#slf4j-api;2.0.7 from central in [default]
org.xerial.snappy#snappy-java;1.1.10.3 from central in [default]
```

conf	modules				artifacts	
	number	search	dwnlded	evicted	number	dwnlded
default	11	11	11	0	11	11

```
at java.base/java.lang.reflect.Method.invoke(Method.java:580)
at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)
at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)
at py4j.Gateway.invoke(Gateway.java:282)
at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)
at py4j.commands.CallCommand.execute(CallCommand.java:79)
at py4j.ClientServerConnection.waitForCommands(ClientServerConnection.java:184)
at py4j.ClientServerConnection.run(ClientServerConnection.java:108)
at java.base/java.lang.Thread.run(Thread.java:1583)
```

```
C:\Users\felip\Downloads\Laboratorio_BigData>SUCCESS: The process with PID 12140 (child process of PID 18328) has been terminated.
SUCCESS: The process with PID 18328 (child process of PID 8684) has been terminated.
SUCCESS: The process with PID 8684 (child process of PID 10480) has been terminated.
```

Capa Batch

Esta capa leerá todos los datos acumulados en Kafka de una sola vez y generará el reporte en formato CSV (conteo total de logs por usuario).

A diferencia de la capa anterior, aquí utilizamos la función “read” tradicional de Spark para extraer todo el historial de eventos almacenados en Kafka en un solo bloque. Este proceso toma la totalidad de los registros, los agrupa por usuario y los ordena de mayor a menor.

Finalmente, el resultado consolidado se escribe y guarda de manera permanente en el disco local como un archivo CSV, cumpliendo la función de almacenamiento histórico de la arquitectura.

Es importante que pandas esté instalado para este proceso: `pip install pandas`

Esta vez usaremos “procesador_batch.py”

```
procesador_batch.py > ...
1  import os
2  os.environ['PYSPARK_SUBMIT_ARGS'] = '--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 pyspark-shell'
3
4  from pyspark.sql import SparkSession
5  from pyspark.sql.functions import from_json, col
6  from pyspark.sql.types import StructType, StructField, StringType, IntegerType
7
8  spark = SparkSession.builder.appName("CapaBatch_Lambda").getOrCreate()
9  spark.sparkContext.setLogLevel("ERROR")
10
11  esquema = StructType([
12      StructField("timestamp", StringType(), True),
13      StructField("usuario", StringType(), True),
14      StructField("accion", StringType(), True),
15      StructField("tiempo_respuesta_ms", IntegerType(), True)
16  ])
17
18  print("Leyendo todos los datos acumulados en Kafka...")
19
20  # Leer desde Kafka en modo Batch (sin readStream)
21  df_batch = spark.read \
22      .format("kafka") \
23      .option("kafka.bootstrap.servers", "localhost:9092") \
24      .option("subscribe", "logs-plataforma") \
25      .load()
26
27  df_parsed = df_batch.selectExpr("CAST(value AS STRING) as json_str") \
28      .select(from_json(col("json_str"), esquema).alias("data")).select("data.*")
29
30  # Generar reporte: Conteo de logs por usuario
31  reporte_csv = df_parsed.groupBy("usuario").count().orderBy(col("count").desc())
32
33  # Mostrar un resumen en consola
34  print("Vista previa del reporte Batch:")
35  reporte_csv.show()
36
37  # Guardar en una carpeta CSV (Capa de Servicio)
38  ruta_salida = "reporte_batch_usuarios"
39  print(f"Guardando reporte en la carpeta: {ruta_salida}...")
40  reporte_csv.write.mode("overwrite").csv(ruta_salida, header=True)
41  print("¡Reporte generado exitosamente!")
```

Magia

Ahora levantamos el Docker y ejecutamos: `python procesador_batch.py`

```
C:\Users\felip\Downloads\Laboratorio_BigData>python procesador_batch.py
WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url = jar:file:/C:/spark/spark-4.2.0-bin-hadoop3/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: C:\Users\felip\.ivy2.5.2\cache
The jars for the packages stored in: C:\Users\felip\.ivy2.5.2\jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-e0d50ac3-93cb-4518-882a-e81ed93c6192;1.0
```

conf	modules					artifacts	
	number	search	dwnlded	evicted		number	dwnlded
default	11	0	0	0		11	0

```
C:\Users\felip\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pyspark\testing\utils.py:127:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
    require_minimum_pandas_version()
Leyendo todos los datos acumulados en Kafka...
Vista previa del reporte Batch:
+-----+
| usuario|count|
+-----+
| Usuario8| 58|
| Usuario6| 47|
| Usuario1| 42|
| Usuario3| 41|
| Usuario9| 41|
| Usuario4| 39|
| Usuario10| 39|
| Usuario5| 37|
| Usuario7| 32|
| Usuario2| 28|
|  Javier| 23|
|  Rayen| 22|
|  Janiera| 21|
|  Esteban| 21|
|  Benji| 20|
|  Shanara| 19|
| Gerardini| 13|
| Benjamin| 12|
|  Amy| 12|
| Berenice| 10|
+-----+
```

```
Guardando reporte en archivo CSV...
C:\Users\felip\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pyspark\sql\pandas\conversion.py:298: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
    require_minimum_pandas_version()
C:\Users\felip\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pyspark\sql\pandas\conversion.py:348: UserWarning: toPandas attempted Arrow optimization because 'spark.sql.execution.arrow.pyspark.enabled' is set to true; however, failed by the reason below:
[PACKAGE_NOT_INSTALLED] PyArrow >= 18.0.0 must be installed; however, it was not found.
Attempting non-optimization as 'spark.sql.execution.arrow.pyspark.fallback.enabled' is set to true.
    warn(msg)
¡Reporte generado exitosamente en el archivo reporte_batch_usuarios.csv!
SUCCESS: The process with PID 14624 (child process of PID 3392) has been terminated.
SUCCESS: The process with PID 3392 (child process of PID 18832) has been terminated.
SUCCESS: The process with PID 18832 (child process of PID 5880) has been terminated.
```

CSV

Cómo resultado final

```
reporte_batch_usuarios.csv X
reporte_batch_usuarios.csv
1  usuario,count
2  Usuario8,58
3  Usuario6,47
4  Usuario1,42
5  Usuario3,41
6  Usuario9,41
7  Usuario4,39
8  Usuario10,39
9  Usuario5,37
10 Usuario7,32
11 Usuario2,28
12 Javiera,23
13 Rayen,22
14 Janiera,21
15 Esteban,21
16 Benji,20
17 Shanara,19
18 Gerardini,13
19 Benjamin,12
20 Amy,12
21 Berenice,10
22
```

Tabla comparativa de arquitecturas Big Data

Criterio	Arquitectura Lambda	Arquitectura Kappa	Data Lake / Data Lakehouse
Idea central	Combina dos rutas de procesamiento (batch y speed) que se unen en una capa de servicio.	Trata todos los datos como un flujo continuo de eventos, con un único pipeline de streaming.	Repositorio centralizado que guarda datos en bruto y los refina por zonas; el Lakehouse suma capacidades de Data Warehouse.
Procesamiento histórico	Sí, mediante una capa batch independiente.	Sí, reprocesando los eventos almacenados en el registro.	Sí, mediante motores batch, consultas SQL y procesamiento distribuido.
Procesamiento en tiempo real	Sí, mediante la capa speed.	Sí, es su enfoque principal.	Sí, cuando se integra con herramientas de streaming.
Organización de los datos	Dataset histórico para batch y vista reciente para speed, unificados en la capa de servicio.	Registro inmutable de eventos (log)	Zonas Raw/Bronze, Cleaned/Silver y Curated/Gold
Tolerancia a fallos y recálculo	La capa batch puede recalcular todo desde los datos originales.	Se reprocesa el log desde el inicio o desde un punto determinado.	Se reprocesa desde la zona Raw, ya que se conservan los datos originales.
Complejidad	Alta, porque hay que mantener dos rutas de procesamiento.	Media a alta, por el dominio técnico que exige el streaming.	Variable, depende del ecosistema, la escala y el nivel de gobierno de datos.
Ventajas	Une análisis histórico confiable con resultados recientes de baja latencia.	Menos duplicación de código y reglas de negocio; lógica centralizada.	Flexible, escalable y económico; soporta BI, ciencia de datos y machine learning.
Desventajas	Lógica duplicada entre capas y riesgo de inconsistencias entre ambas.	Reprocesar historiales grandes requiere planificación y capacidad; poco conveniente para análisis batch extensos.	Sin gobierno de datos puede degenerar en un <i>data swamp</i> .

Tecnologías típicas	Kafka, Spark (batch y streaming), SQLite o similar como capa de servicio.	Apache Kafka, Apache Flink o Spark Streaming.	Delta Lake, Apache Iceberg, Apache Hudi, Spark, S3 / Azure Data Lake / Google Cloud Storage.
Mejor escenario de uso	Sistemas que necesitan análisis histórico y resultados recientes a la vez.	Sistemas basados en eventos continuos, con monitoreo y respuesta inmediata.	Plataformas integrales de datos, analítica, BI e inteligencia artificial.
Ejemplos de aplicación	Analítica operacional, reportes históricos, monitoreo actualizado.	Detección de fraude, telemetría, IoT, monitoreo financiero.	Centralización de datos, dashboards, modelos predictivos.

Conclusión

En este laboratorio se logró implementar una simulación funcional de la arquitectura Lambda en un entorno local. Al integrar Apache Kafka mediante Docker y utilizar PySpark, construimos un flujo capaz de procesar datos en dos vías de manera simultánea.

Por un lado, la capa Batch tomó todos los eventos acumulados y los consolidó en un reporte CSV. Por otro lado, la capa Speed nos demostró cómo se pueden ir leyendo y procesando las acciones de los usuarios en tiempo real desde la consola.

Para concluir, el trabajo cumplió su objetivo, ya que pudimos comprobar cómo es que esta arquitectura logra un buen equilibrio entre reaccionar rápido a los datos nuevos y mantener un registro histórico preciso.

Bibliografía

Universidad Autónoma de Chile (2026). *Material Complementario Arquitecturas Big Data*. Módulo Semana 08, Asignatura Big Data.

Python Software Foundation (2026). *Download Python*. Recuperado de:
<https://www.python.org/downloads/>.

Oracle (2026). *Java SE Development Kit 21 Downloads*. Recuperado de:
<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html>

Docker Inc. (2026). *Docker Desktop*. Recuperado de:
<https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe>.

Apache Software Foundation (2026). *Apache Spark Downloads*. Recuperado de:
<https://spark.apache.org/downloads.html>

cdarlint (2026). *winutils: winutils.exe hadoop.dll and hdfs.dll binaries for hadoop windows*. GitHub